LEKCIJA 3

Softver

Od operativnog sistema do web, mobilnih, cloud i AI aplikacija

Cilj lekcije: Razumjeti ulogu softvera u računarskom sistemu, razlikovati sistemski i aplikativni softver, objasniti ključne funkcije operativnog sistema te prepoznati savremene modele isporuke, licenciranja, ažuriranja i sigurnog korištenja softvera.

Ishodi učenja

- razlikovati sistemski, aplikativni i razvojni softver te objasniti njihovu međusobnu zavisnost;
- objasniti ključne funkcije operativnog sistema: upravljanje procesima, memorijom, datotekama, uređajima i sigurnošću;
- opisati savremeni proces pokretanja računara i ulogu firmwarea, UEFI-a, boot managera i kernela;
- razlikovati desktop, web, mobilne, SaaS i cloud-native aplikacije;
- razumjeti osnovne modele licenciranja i distribucije softvera, uključujući proprietary i open-source modele;
- objasniti zašto ažuriranja, upravljanje zavisnostima i siguran razvoj predstavljaju dio životnog ciklusa softvera;
- prepoznati kako generativna AI i ugrađene AI funkcije mijenjaju savremene aplikacije.

Napomena: Razvoj softvera, cloud computing, virtuelizacija i informacijska sigurnost obrađuju se detaljno u posebnim nastavnim jedinicama. Ovdje se uvode samo koncepti potrebni za razumijevanje savremenog softverskog okruženja.

1. Šta je softver?

Softver predstavlja skup programa, podataka, konfiguracija i pripadajućih komponenti koje računarskom sistemu daju funkcionalnost. Za razliku od hardvera, koji čine fizičke komponente, softver opisuje instrukcije i pravila prema kojima hardver obrađuje podatke, komunicira s drugim uređajima i pruža usluge korisniku.

U najjednostavnijem modelu korisnik ne komunicira direktno s procesorom, memorijom, diskovima ili mrežnim adapterom. Aplikacija se oslanja na operativni sistem, operativni sistem koristi upravljačke programe i firmware, a oni upravljaju fizičkim uređajima. Zbog toga je softver najbolje posmatrati kao slojeviti sistem, a ne kao kolekciju nepovezanih programa.

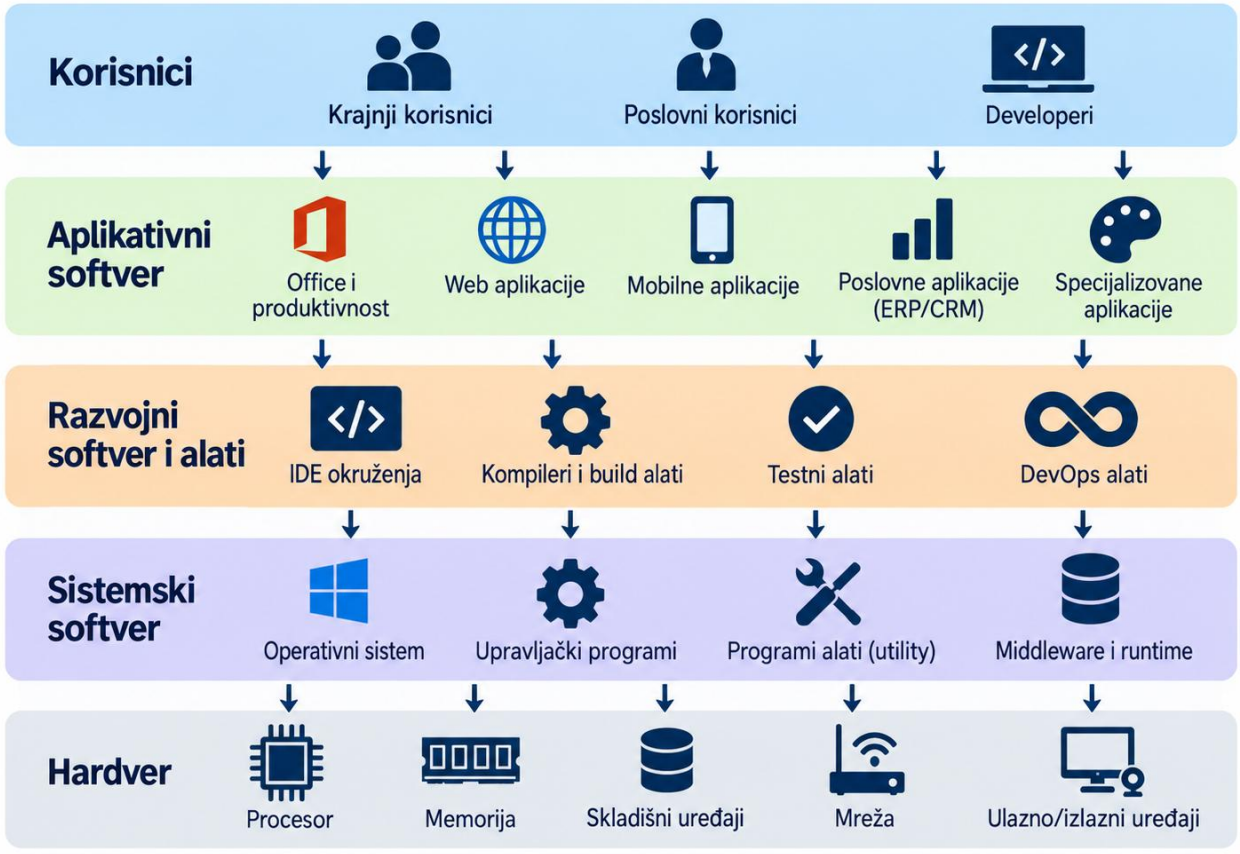
Ključna ideja: Hardver daje računsku sposobnost, ali softver određuje kako će se ta sposobnost koristiti. Vrijednost računarskog sistema nastaje njihovim zajedničkim radom.

1.1. Osnovna podjela softvera

Kategorija	Osnovna uloga	Primjeri
Sistemski softver	Upravlja hardverom i pruža osnovne usluge drugim programima.	Operativni sistem, device driver, sistemski utility, firmware.

Kategorija	Osnovna uloga	Primjeri
Aplikativni softver	Rješava konkretne korisničke ili poslovne zadatke.	Browser, uredski paket, ERP, CAD, mobilna aplikacija.
Razvojni softver	Omogućava izradu, testiranje i održavanje drugih programa.	IDE, compiler, debugger, version control alati.
Middleware i runtime	Povezuje komponente ili pruža okruženje za izvršavanje aplikacija.	Application server, .NET/JVM runtime, message broker.

Granice između kategorija nisu uvijek stroge. Browser je aplikacija, ali istovremeno predstavlja izvršno okruženje za veliki broj web aplikacija. Cloud platforma može pružati sistemske funkcije kao uslugu, dok mobilni operativni sistem uključuje i veliki broj aplikativnih komponenti. Zbog toga je važnije razumjeti ulogu softvera nego insistirati na jednoj apsolutnoj klasifikaciji.



Slika 1. Slojevi softvera i njihov odnos prema korisnicima i hardveru

2. Sistemski softver

Sistemski softver predstavlja sloj između aplikacija i hardvera. Njegov najvažniji dio je operativni sistem, ali u ovu grupu se obično uključuju i upravljački programi, firmware i sistemski utility programi. Zadatak ovog sloja je da sakrije veliki dio hardverske složenosti i aplikacijama ponudi standardizovane usluge.

2.1. Operativni sistem

Operativni sistem (OS) upravlja računarskim resursima i koordinira izvršavanje programa. Savremeni operativni sistemi obavljaju mnogo više od prikazivanja korisničkog interfejsa: upravljaju procesima i threadovima, fizičkom i virtuelnom memorijom, datotečnim sistemima, uređajima, mrežom, identitetima, pravima pristupa i brojnim servisima u pozadini.

Funkcija OS-a	Šta se upravlja	Zašto je važno
Procesi i CPU	Procesi, threadovi, prioriteti i raspoređivanje.	Više programa dijeli procesorsko vrijeme bez međusobnog direktnog upravljanja procesorom.
Memorija	RAM, virtual memory, zaštita adresnog prostora.	Programi dobijaju izolovan prostor i kontrolisan pristup memoriji.
Datoteke	Datoteke, direktoriji, metadata, dozvole i storage uređaji.	Podaci se organizuju, pronalaze, štite i trajno pohranjuju.
Uređaji	Driveri, ulazno/izlazni zahtjevi, prekidi.	Aplikacije koriste standardne interfejse umjesto specifičnih detalja svakog uređaja.
Mreža	Mrežni protokoli, interfejsi, socketi i firewall funkcije.	Aplikacije mogu komunicirati lokalno i preko Interneta.
Sigurnost	Identitet, autentikacija, autorizacija, izolacija i audit.	Ograničava ko i šta može koristiti pojedine resurse.

3. Pokretanje računara: od firmwarea do operativnog sistema

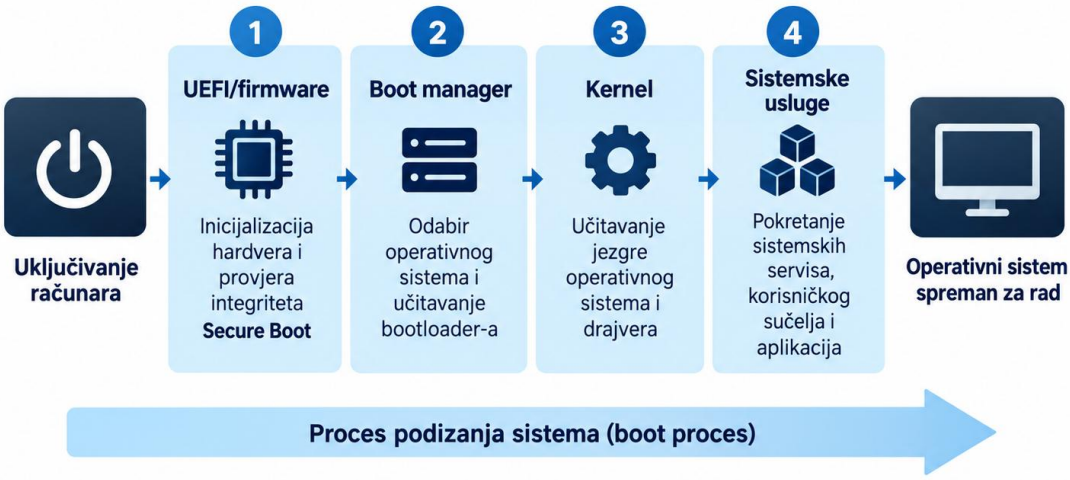
Stariji materijali često opisuju pokretanje računara isključivo kroz BIOS. Savremeni PC sistemi u pravilu koriste UEFI (Unified Extensible Firmware Interface), koji predstavlja moderniji firmware interfejs i podržava funkcije kao što je Secure Boot. Microsoft, na primjer, za Windows 11 navodi UEFI firmware i mogućnost korištenja Secure Boota kao dio sistemskih zahtjeva.1

Korak	Šta se dešava
1. Uključivanje	Procesor započinje izvršavanje firmware koda i inicijalizuje osnovni hardver.
2. Firmware / UEFI	Provjeravaju se uređaji, konfiguracija i dostupni boot izvori.
3. Boot manager	Odabire se operativni sistem ili odgovarajuća boot konfiguracija.
4. Kernel	Kernel OS-a se učitava u memoriju i preuzima upravljanje procesorom, memorijom i uređajima.
5. Sistemski servisi	Pokreću se osnovni servisi, sigurnosne komponente, mreža i korisničko okruženje.
6. Prijava korisnika	Korisnik dobija interaktivno okruženje i može pokretati aplikacije.

Savremeni kontekst: Secure Boot nije “antivirus pri paljenju računara”. Njegova uloga je pomoći da se

1 Microsoft Support, Windows 11 System Requirements: system firmware treba biti UEFI i Secure Boot capable; pristupljeno 2026.

tokom boot procesa pokreću povjerljive, kriptografski potpisane komponente, čime se smanjuje mogućnost ranog preuzimanja sistema od neautorizovanog koda.



Slika 2. Savremeni boot proces (od UEFI do operativnog sistema)

4. Upravljanje procesima i memorijom

Korisnik može istovremeno imati otvoren browser, videokonferenciju, editor teksta, antivirus, cloud sync i desetine pozadinskih servisa. Operativni sistem zato mora raspoređivati procesorsko vrijeme i memoriju tako da programi koegzistiraju bez direktnog konflikta.

4.1. Procesi, threadovi i multitasking

Proces je instanca programa u izvršavanju. Jedan proces može imati jedan ili više threadova, odnosno tokova izvršavanja. Scheduler operativnog sistema dodjeljuje procesorsko vrijeme spremnim threadovima i prebacuje CPU između njih. Na višejezgrenim procesorima više threadova može se izvršavati zaista paralelno, dok se ostali brzo smjenjuju prema prioritetima i raspoloživosti resursa.

Pojam multitasking zato danas nije posebna luksuzna karakteristika operativnog sistema, nego osnovna pretpostavka općih desktop, mobilnih i serverskih OS-ova. Multiuser sistemi dodatno izoliraju korisničke sesije i primjenjuju zasebna prava pristupa.

4.2. Memorija, queue, buffer i spool

Upravljanje memorijom obuhvata dodjelu i oslobađanje RAM-a, zaštitu memorijskog prostora procesa i korištenje virtualne memorije. Kada fizičke memorije nema dovoljno ili operativni sistem želi efikasnije upravljati memorijom, dijelovi sadržaja mogu se privremeno premještati između RAM-a i sekundarne pohrane.

Pojam	Precizno značenje	Primjer
Queue	Struktura reda čekanja; često se obrađuje principom FIFO, zavisno od algoritma.	Red poslova koji čekaju obradu.
Buffer	Privremeni prostor za podatke dok se prenose ili obrađuju između komponenti različite brzine.	Streaming audio/video, mrežni prijenos.
Spool	Privremeno pohranjivanje poslova za uređaj ili servis koji ih obrađuje svojim tempom.	Print spooler i red dokumenata za štampanje.

5. Datotečni sistemi i upravljanje uređajima

Datotečni sistem organizuje trajno pohranjene podatke u datoteke i direktorije. Operativni sistem vodi metadata informacije kao što su naziv, veličina, vrijeme izmjene, vlasništvo i prava pristupa. Hijerarhijska organizacija direktorija ostaje osnovni mentalni model, iako korisnik danas sadržaj često pronalazi kroz pretragu, biblioteke, cloud sync i aplikacijske interfejse umjesto ručnog prolaska kroz foldere.

Na različitim platformama koriste se različiti datotečni sistemi i modeli putanja. Windows tradicionalno koristi oznake diskova kao što je C:, dok Unix/Linux sistemi koriste jedinstveno stablo direktorija sa root direktorijem “/”. Za uvodni predmet važnije je razumjeti zajedničke principe: hijerarhiju, putanju, metadata, dozvole, kapacitet, pouzdanost i backup.

5.1. Upravljački programi i firmware

Device driver je softverska komponenta koja omogućava operativnom sistemu da komunicira s konkretnim hardverskim uređajem. Driver prevodi opće zahtjeve operativnog sistema u operacije koje razumije uređaj. Firmware je softver pohranjen u samom uređaju ili njegovoj nevolatilnoj memoriji i upravlja osnovnim ponašanjem uređaja. U praksi se firmware i driveri također ažuriraju, naročito radi kompatibilnosti, stabilnosti i sigurnosti.

6. Operativni sistemi danas

Umjesto jedne tržišne tabele koja brzo zastarijeva, korisnije je posmatrati operativne sisteme prema tipu uređaja i ulozi. Danas paralelno postoje desktop, serverski, mobilni, embedded i specijalizovani operativni sistemi, a jedan korisnik tokom dana često koristi više različitih OS platformi.

Okruženje	Primjeri porodica	Tipična uloga
Desktop/laptop	Windows, macOS, Linux distribucije	Opća produktivnost, razvoj, kreativni i profesionalni rad.
Server	Linux/Unix porodice, Windows Server	Web, baze podataka, identitet, aplikacijski i infrastrukturni servisi.
Mobilni uređaji	Android, iOS/iPadOS	Mobilne aplikacije, komunikacija, senzori, digitalni servisi.
Embedded/IoT	Embedded Linux, RTOS i vendor-specifični sistemi	Uređaji posebne namjene, industrija, automobili, senzori.
Cloud/containers	Host OS + container runtime / orchestration sloj	Izvršavanje velikog broja izolovanih aplikacijskih komponenti.

Virtuelizacija i kontejneri dodatno mijenjaju odnos između aplikacije i operativnog sistema. Cloud-native pristupi često koriste kontejnere, mikroservise i deklarativne API-je kako bi aplikacije bile skalabilnije i prenosivije između modernih cloud okruženja.2

7. Aplikativni softver

Aplikativni softver je namijenjen izvođenju zadataka koji imaju direktnu vrijednost za korisnika ili organizaciju. Umjesto nabrajanja pojedinačnih proizvoda, korisnije je posmatrati kategorije poslova koje aplikacije podržavaju.

Kategorija	Primjeri funkcija
Produktivnost i saradnja	Obrada teksta, tabele, prezentacije, e-mail, chat, videokonferencije i zajednički rad.
Poslovni sistemi	ERP, CRM, HRIS, finansije, prodaja, logistika i workflow.
Kreativni i medijski alati	Grafika, fotografija, audio/video obrada, animacija i publishing.
Inženjerski i naučni softver	CAD/CAM, simulacije, statistika, modeliranje i specijalizovana analiza.
Podaci i analitika	Baze podataka, BI, vizualizacija, data science i izvještavanje.
Komunikacija i web	Browseri, komunikacijski klijenti, web aplikacije i online platforme.
Razvojni alati	Editor/IDE, compiler, debugger, version control i testiranje.
Sigurnosni alati	Endpoint zaštita, enkripcija, upravljanje identitetima, monitoring i backup.
AI-enabled aplikacije	Generisanje sadržaja, asistenti, preporuke, pretraga i automatizacija zasnovana na modelima.

8. Kako se aplikacije danas isporučuju

Klasična slika aplikacije kao programa koji se instalira s fizičkog medija više nije dovoljna. Savremeni softver se isporučuje kroz više modela, a granice među njima se često preklapaju.

8.1. Desktop i native aplikacije

Native aplikacija izvršava se neposredno na ciljnoj platformi i obično ima direktniji pristup lokalnom hardveru i funkcijama operativnog sistema. Takve aplikacije i dalje dominiraju kada su potrebne visoke performanse, rad bez mreže, specijalizovani hardver ili duboka integracija sa desktop okruženjem.

8.2. Web aplikacije

Web aplikacije koriste browser kao korisničko i izvršno okruženje. Dio logike može se izvršavati na klijentu, dio na serveru, a podaci se razmjenjuju preko mrežnih protokola i API-ja. W3C opisuje web standarde kao

2 Cloud Native Computing Foundation (CNCF), Cloud Native Definition / Cloud Native Technology: cloud-native tehnologije uključuju kontejnere, mikroservise i deklarativne API-je, te ciljaju skalabilne i upravljive aplikacije.

temelj otvorene web platforme za razvoj aplikacija dostupnih na različitim uređajima; HTML, CSS i širok skup web API-ja dio su tog ekosistema.3

Zbog toga formulacija “web aplikacija samo isporučuje sadržaj sa servera klijentu” više nije dovoljna. Moderne web aplikacije mogu imati kompleksan interfejs, lokalno izvršavanje, offline funkcije, real-time komunikaciju i pristup određenim mogućnostima uređaja, zavisno od browsera i sigurnosnih dozvola.

8.3. Mobilne aplikacije

Savremeni mobilni ekosistem dominantno se zasniva na Android i Apple platformama. Aplikacije mogu biti potpuno native, cross-platform ili kombinovati web i native komponente. Mobilni OS ograničava aplikacije sandboxingom, sistemom dozvola i pravilima distribucije, što utiče na arhitekturu, privatnost i sigurnost aplikacija. Googleove smjernice za Android, na primjer, preporučuju slojevitu arhitekturu i separation of concerns radi kvaliteta, održavanja i testiranja aplikacija.4

Distribucija mobilnog softvera često se odvija preko upravljanih prodavnica aplikacija. Apple navodi App Store kao kanal za objavu i distribuciju aplikacija, uz procese pregleda, analitiku i upravljanje aplikacijskim sadržajem.5

8.4. SaaS i cloud-native softver

Kod Software-as-a-Service modela korisnik ne mora upravljati lokalnom instalacijom cijele aplikacije. Aplikacija se koristi kao usluga, najčešće kroz browser ili klijentsku aplikaciju, dok infrastrukturu, deployment i veliki dio održavanja vodi pružalac usluge. Cloud-native aplikacije idu korak dalje u dizajnu: koriste automatizaciju, kontejnere, API-je i distribuirane komponente kako bi bolje iskoristile elastičnost i otpornost modernih cloud platformi.6

3 World Wide Web Consortium (W3C), Web Standards: otvorena web platforma koristi standarde kao što su HTML, CSS, SVG, WebRTC i različiti web API-ji.

4 Android Developers, Recommendations for Android architecture: preporučuje se layered architecture i separation of concerns radi kvaliteta, skalabilnosti, održavanja i testiranja.

5 Apple Developer, App Store / Distribution: App Store je kanal za distribuciju aplikacija na Apple platformama; dokumentacija opisuje pregled, objavu, pretplate i distribuciju.

6 CNCF, Cloud Native Technology: cloud-native tehnologije pomažu izgradnji skalabilnih, otpornijih i upravljivih aplikacija u javnim, privatnim i hibridnim cloud okruženjima.



Slika 3. Modeli isporuke i pokretanja aplikativnog softvera

9. Licenciranje, vlasništvo i način plaćanja

Jedna od čestih grešaka u starijim podjelama softvera jeste miješanje pravnog statusa, prava nad izvornim kodom i poslovnog modela u jednu listu. Pojmovi commercial, freeware, shareware, open source, subscription i public domain ne pripadaju istoj dimenziji klasifikacije.

Dimenzija	Primjeri	Šta opisuje
Prava/licenca	Proprietary, open source, public domain	Koja prava korisnik ima nad korištenjem, kopiranjem, modifikacijom i distribucijom.
Cijena/model plaćanja	Besplatno, jednokratna kupovina, pretplata, freemium	Kako se finansira pravo korištenja ili usluga.
Način isporuke	Lokalna instalacija, app store, web/SaaS	Kako korisnik dobija i koristi softver.
Dostupnost koda	Source available, open source, closed source	Da li je izvorni kod dostupan i pod kojim uslovima.

Open source ne znači jednostavno “besplatan softver”. Open Source Initiative naglašava da open-source softver mora biti distribuiran pod licencom koja ispunjava kriterije Open Source Definition i omogućava

korištenje, izmjenu i redistribuciju u skladu s licencom. Takav softver se može koristiti i u komercijalne svrhe.7

Važna razlika: Freeware znači da se softver može koristiti bez naknade prema uslovima autora; to ne znači da korisnik automatski dobija izvorni kod ili pravo izmjene. Open source opisuje licencna prava, a ne nužno cijenu.

10. Instalacija, distribucija i životni ciklus softvera

Softver nije statičan proizvod. Nakon inicijalne instalacije slijede konfiguracija, ažuriranja, zakrpe, promjene verzija, migracije podataka i na kraju prestanak podrške. Upravljanje tim životnim ciklusom posebno je važno u organizacijama koje koriste stotine ili hiljade uređaja i aplikacija.

- Instalacija i provisioning: postavljanje aplikacije i početne konfiguracije.
- Update: funkcionalne, kompatibilnosne ili sigurnosne izmjene postojeće verzije.
- Patch: ciljana korekcija greške ili ranjivosti.
- Upgrade: prelazak na novu glavnu verziju ili platformu.
- Rollback: vraćanje na prethodno stabilno stanje kada nova verzija uzrokuje problem.
- End of support / end of life: trenutak nakon kojeg proizvođač više ne isporučuje redovne zakrpe ili podršku.

U modernim sistemima distribucija se sve više automatizuje preko package managera, app storeova, enterprise management sistema i CI/CD mehanizama. Ovaj model povećava brzinu promjena, ali istovremeno povećava važnost kontrole verzija, testiranja i mogućnosti vraćanja na prethodno stanje.



Slika 4. Životni ciklus razvoja softvera i ključne sigurnosne prakse

7 Open Source Initiative (OSI), Frequently Answered Questions i Open Source Definition: open-source softver se može koristiti, mijenjati i dijeliti prema uslovima OSI-kompatibilne licence; komercijalna upotreba je dozvoljena.

11. Softver i sigurnost

Softver istovremeno predstavlja sigurnosnu kontrolu i potencijalnu površinu napada. Firewall, enkripcija, autentikacija i endpoint zaštita implementiraju se kroz softverske komponente, ali ranjivost u istim ili drugim programima može omogućiti napadaču zaobilaženje kontrola. Zbog toga se sigurnost ne može dodati samo na kraju razvoja.

NIST Secure Software Development Framework (SSDF) preporučuje integrisanje sigurnosnih praksi u životni ciklus razvoja kako bi se smanjio broj ranjivosti u objavljenom softveru, ublažio uticaj neotkrivenih ranjivosti i adresirali njihovi uzroci.8

Sigurnosna tema	Primjer problema	Osnovna mjera
Ranjivosti	Greška omogućava neovlašteno izvršavanje ili pristup.	Patch management, testiranje, secure coding.
Identitet i prava	Aplikacija ima više privilegija nego što joj je potrebno.	Least privilege i jaka autentikacija.
Zavisnosti	Aplikacija koristi ranjivu biblioteku ili paket.	Inventory/SBOM, praćenje verzija i ažuriranja.
Distribucija	Lažna ili izmijenjena instalacija.	Code signing, provjera integriteta i pouzdani kanali.
Podaci	Aplikacija nepotrebno prikuplja ili izlaže podatke.	Data minimization, enkripcija i kontrola pristupa.
Konfiguracija	Sigurna funkcija postoji, ali nije uključena ili je pogrešno podešena.	Secure defaults i hardening.

CISA-in pristup Secure by Design dodatno naglašava da proizvođači trebaju preuzeti veću odgovornost za sigurnosne ishode i omogućiti sigurne postavke kao podrazumijevane, umjesto da se cjelokupan teret zaštite prebacuje na krajnjeg korisnika.9

11.1. Maliciozni softver nije isto što i ranjiv softver

Malware je softver napravljen s namjerom da nanese štetu, omogući neovlašten pristup, krađu podataka ili druge zlonamjerne aktivnosti. Virusi, crvi, trojanci, ransomware i spyware predstavljaju različite kategorije ili obrasce ponašanja. Ranjiv softver, nasuprot tome, može biti legitiman program koji sadrži grešku koju napadač može zloupotrijebiti. Ova razlika je važna: sigurnost zahtijeva i detekciju zlonamjernog koda i smanjenje ranjivosti legitimnog softvera.

12. AI kao dio savremenog softvera

Umjetna inteligencija se sve rjeđe pojavljuje samo kao zasebna aplikacija. AI funkcionalnosti ugrađuju se u uredske alate, pretraživače, razvojna okruženja, grafičke programe, CRM sisteme, analitičke platforme i korisničku podršku. Generativni modeli omogućavaju rad sa prirodnim jezikom, kodom, slikom, zvukom i drugim tipovima sadržaja.

---

8 NIST SP 800-218, Secure Software Development Framework (SSDF) Version 1.1, 2022; NIST je 2025. objavio initial public draft revizije 1 / verzije 1.2.

9 CISA, Secure by Design guidance: sigurnost treba biti osnovna poslovna i tehnička osobina proizvoda, a sigurne postavke trebaju biti što je moguće više podrazumijevane.

Za korisnika je važno razumjeti da AI komponenta ne mijenja osnovne zahtjeve kvalitetnog softvera. Aplikacija i dalje mora imati pouzdane podatke, kontrolu pristupa, ažuriranja, jasne dozvole i provjerljive procese. AI dodatno uvodi pitanja tačnosti generisanih odgovora, zaštite podataka, transparentnosti i potrebe za ljudskom provjerom rezultata.

Princip: AI-enabled software nije automatski inteligentniji, sigurniji ili tačniji od klasičnog softvera. Vrijednost zavisi od kvaliteta modela, podataka, integracije, konteksta i načina na koji korisnik provjerava rezultat.

13. Kako se softver promijenio

Poređenje starijeg i savremenog softverskog okruženja pokazuje nekoliko velikih promjena. Fizički mediji zamijenjeni su online distribucijom; pojedinačne desktop aplikacije dopunjene su web i mobilnim servisima; licence sa jednokratnim plaćanjem često koegzistiraju s pretplatama; aplikacije koriste veliki broj vanjskih biblioteka i API-ja; cloud omogućava globalnu isporuku, a AI postaje ugrađena funkcija brojnih proizvoda.

Raniji dominantni model	Savremeni dominantni trend
Softver na CD/DVD mediju	Digitalna distribucija, app storeovi i package manageri.
Samostalna desktop aplikacija	Web, mobilne, SaaS i povezane multi-device aplikacije.
Rijetka velika ažuriranja	Kontinuirana, inkrementalna ažuriranja i automatizovani deployment.
Aplikacija kao monolit	API integracije, servisi i cloud-native komponente kada je opravdano.
Funkcije potpuno programirane pravilima	Kombinacija determinističkog koda i AI/ML komponenti.
Sigurnost kao dodatak	Secure-by-design i sigurnost kroz cijeli životni ciklus.

14. Zaključak

Softver je sloj koji pretvara računarske resurse u funkcionalne sisteme. Operativni sistem koordinira hardver i pruža zajedničke usluge aplikacijama, dok aplikativni softver rješava konkretne potrebe korisnika i organizacija. Savremeni softver više nije ograničen na program instaliran na jednom računaru: izvršava se u browseru, na mobilnom uređaju, u cloudu, u kontejnerima i unutar povezanih distribuiranih sistema.

Najvažniji pomak za studenta uvodnog IT predmeta jeste razumjeti softver kao sistem sa životnim ciklusom. Njegova vrijednost ne zavisi samo od funkcionalnosti, nego i od kompatibilnosti, održavanja, licenciranja, distribucije, kvaliteta, sigurnosti i načina na koji se integriše sa drugim servisima i podacima.

Poruka lekcije: Pitanje nije samo “koji program koristimo?”, nego: na kojem sloju radi, kako se isporučuje i održava, koja prava daje korisniku, od čega zavisi i kako se štiti.

Ključni pojmovi

- software • system software • application software • firmware • device driver
- operating system (OS) • kernel • process • thread • scheduler • multitasking
- virtual memory • buffer • queue • spool • file system • permissions
- UEFI • Secure Boot • native application • web application • mobile application
- SaaS • cloud-native • API • open source • proprietary software • freeware

- software update • patch • dependency • code signing • secure by design • malware • AI-enabled software

Pitanja za provjeru znanja

1. Koja je osnovna razlika između sistemskog i aplikativnog softvera?
2. Koje su najvažnije funkcije operativnog sistema?
3. Objasni ulogu UEFI-a, boot managera i kernela tokom pokretanja računara.
4. Koja je razlika između procesa i threada?
5. Šta je virtualna memorija i zašto se koristi?
6. Objasni razliku između queue, buffer i spool pojmova.
7. Koja je uloga device drivera, a koja firmwarea?
8. Zašto tržišni udio operativnih sistema nije dobar trajni kriterij za klasifikaciju OS-ova?
9. Koja je razlika između desktop/native, web i mobilne aplikacije?
10. Zašto open source nije isto što i freeware?
11. Kako se pretplatnički model razlikuje od vrste licence?
12. Zašto su update i patch dio sigurnosti softvera?
13. Koja je razlika između malwarea i legitimnog softvera koji sadrži ranjivost?
14. Šta znači secure by design?
15. Koje nove zahtjeve uvodi AI-enabled softver?

Istraživački izazovi

16. Odaberi jednu desktop aplikaciju koju koristiš. Identificiraj operativni sistem, runtime, zavisnosti i način na koji prima ažuriranja.
17. Uporedi isti tip aplikacije u desktop, web i mobilnoj verziji. Analiziraj šta se izvršava lokalno, šta na serveru i koje mogućnosti uređaja aplikacija koristi.
18. Izaberi jedan open-source proizvod i pronađi njegovu licencu. Objasni koja prava i obaveze licenca daje korisniku.
19. Na jednom savremenom računaru ili telefonu pronađi najmanje pet sistemskih procesa ili servisa i objasni njihovu ulogu.
20. Odaberi aplikaciju koja koristi generativnu AI. Analiziraj koje podatke korisnik šalje, koji rezultat dobija i gdje je potrebna ljudska provjera.
21. Pronađi primjer proizvoda koji je izašao iz podrške (end of life). Objasni zašto nastavak korištenja može biti poslovni i sigurnosni rizik.

Literatura i izvori

1. NIST. (2022). SP 800-218: Secure Software Development Framework (SSDF) Version 1.1. National Institute of Standards and Technology.
2. NIST. (2025). SP 800-218 Rev. 1 Initial Public Draft: Secure Software Development Framework Version 1.2.
3. CISA. Secure by Design. Cybersecurity and Infrastructure Security Agency.
4. Open Source Initiative. The Open Source Definition; Frequently Answered Questions; OSI Approved Licenses.
5. World Wide Web Consortium (W3C). Web Standards.
6. Cloud Native Computing Foundation (CNCF). Cloud Native Definition / Cloud Native Technology.
7. Microsoft Support. Windows 11 System Requirements - UEFI and Secure Boot.

8. Android Developers. Recommendations for Android Architecture.

9. Apple Developer. App Store and Distribution documentation.

10. Silberschatz, A., Galvin, P. B., & Gagne, G. Operating System Concepts. Wiley.

11. Tanenbaum, A. S., & Bos, H. Modern Operating Systems. Pearson.